

MICROSAR Classic Mem_30_LegacyAdapter

Technical Reference

Contents

1	Introduction	5
1.1	Architecture Overview	5
2	Functional Description	6
2.1	Features	6
2.1.1	Adapter to Legacy Memory Drivers	6
2.1.2	Hardware Specific Services	6
2.1.3	Multi Memory Device Instance Support	6
2.1.4	Service Invocation	6
2.2	Error Handling	6
2.2.1	Development Error Reporting	6
2.3	Compliance	7
3	Integration	8
3.1	Embedded Implementation	8
3.2	Initialization	8
3.3	Main Functions	9
3.4	Dynamic activation	9
3.5	Synchronization	10
3.5.1	Critical Sections	11
3.6	Memory Sections	11
3.7	Callouts	11
3.8	Error Reporting Det	11
4	API Description	12
4.1	Provided Functions	12
4.1.1	Mem_30_LegacyAdapter_BlankCheck	13
4.1.2	Mem_30_LegacyAdapter_DelInit	14
4.1.3	Mem_30_LegacyAdapter_Erase	15
4.1.4	Mem_30_LegacyAdapter_GetJobResult	16
4.1.5	Mem_30_LegacyAdapter_GetVersionInfo	17
4.1.6	Mem_30_LegacyAdapter_HwSpecificService	18
4.1.7	Mem_30_LegacyAdapter_Init	19
4.1.8	Mem_30_LegacyAdapter_MainFunction	20
4.1.9	Mem_30_LegacyAdapter_PropagateError	21
4.1.10	Mem_30_LegacyAdapter_Read	22
4.1.11	Mem_30_LegacyAdapter_Resume	23
4.1.12	Mem_30_LegacyAdapter_Suspend	24

- 4.1.13 Mem_30_LegacyAdapter_Write 25
 - 4.2 Required Functions 26
 - 4.3 Service Ports 26
- 5 Configuration 27
 - 5.1 Configuration Variants 27
 - 5.2 Additional Configuration Tasks 27
- 6 Glossary and Abbreviations 29
 - 6.1 Glossary 29
 - 6.2 Abbreviations 29
- 7 Contact 30

Document Information

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_MemoryDriver.pdf	R23-11
[2]	AUTOSAR	AUTOSAR_SWS_DevelopmentErrorTracer.pdf	4.xx.xx
[3]	AUTOSAR	AUTOSAR_SWS_MemoryAccess.pdf	R23-11
[4]	Vector	TechnicalReference_MemAcc.pdf	see delivery
[5]	AUTOSAR	AUTOSAR_SWS_FlashDriver.pdf	R23-11
[6]	AUTOSAR	AUTOSAR_SWS_EEPROMDriver.pdf	R23-11



Caution

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

1 Introduction

This document describes the functionality, integration, API and configuration of the module Mem_30_LegacyAdapter.

- > The Mem driver is specified in AUTOSAR [1].
- > The Flash driver is specified in AUTOSAR [5].
- > The EEPROM driver is specified in AUTOSAR [6].

The Mem driver's (Mem_30_LegacyAdapter) task is the low level memory access based on the physical segmentation of the underlying memory device technology. The API of the Mem driver is memory device technology independent and thus provides a memory device agnostic interface to the Memory Access Module upper layer.

This module is responsible for the following:

- > Providing basic services for accessing the flash memory like read, write, erase and blank check.
- > Enabling execution of services for different memory instances.



Note

The feature set in this implementation only available together with legacy memory drivers such as vMem, Flash or EEPROM drivers. This implementation only acts as an adapter to the AUTOSAR specified Mem module.

1.1 Architecture Overview

These are the interfaces of the module:

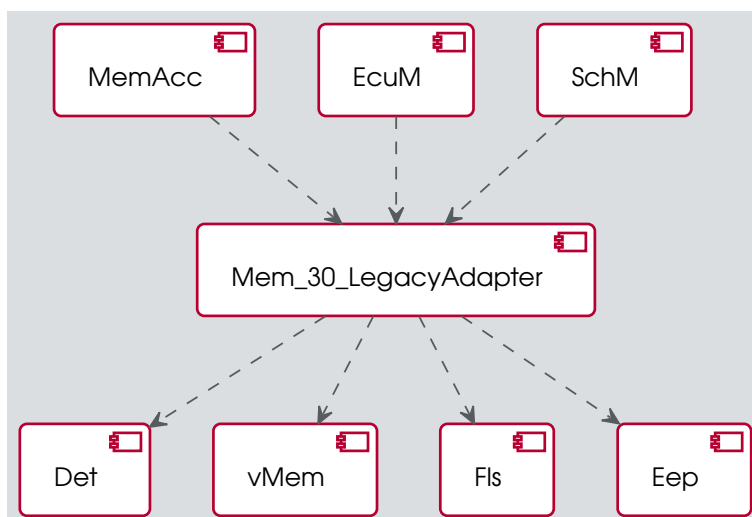


Figure 1.1 Module Architectural Environment

2 Functional Description

2.1 Features

The features described below cover the main functionality specified for this module.

2.1.1 Adapter to Legacy Memory Drivers

The main functionality of this module is acting as an adapter for legacy memory drivers. Currently, vMem, Flash and EEPROM drivers are supported. For more information regarding the functionality of the legacy memory driver, refer to their respective Technical Reference document.



Caution

While it is possible to reference multiple different legacy drivers at the same time, it is crucial that they all have the same erased value and that this value is correctly configured in the *MemPublishedInformation/MemErasedValue* as upper layers rely on this value.

2.1.2 Hardware Specific Services

The adapter supports the mapping of the hardware-specific service of the legacy memory driver (vMem) to the api of an AUTOSAR Mem driver. The ids of the hardware-specific functionality in the function pointer table of the vMem driver is mapped to the hardware service id of the AUTOSAR api. Please, refer to the Technical Reference document of the used vMem drivers for more information regarding their hardware-specific services. In case the underlying driver is a Flash or EEPROM driver, no hardware-specific service is provided.

2.1.3 Multi Memory Device Instance Support

The Mem_30_LegacyAdapter supports multiple instances across multiple underlying drivers.

2.1.4 Service Invocation

The Mem_30_LegacyAdapter provides only indirect service invocation (INDIRECT_STATIC as well as INDIRECT_DYNAMIC).

2.2 Error Handling

2.2.1 Development Error Reporting

The module supports development error detection and reporting using the AUTOSAR Det. If enabled, these errors are reported using the service `Det_ReportError()` as specified in [2].

The reported service IDs and error IDs are contained and documented in the module Types header file.

2.3 Compliance

These are the major **unsupported** features compared to AUTOSAR [1], as no legacy driver supports them:

- > Direct static service invocation - DIRECT_STATIC
- > Suspend and Resume
- > Delnit

Other features might exist, that a specific underlying legacy driver supports. Please refer to the technical reference of the used driver for more information regarding the capabilities of the driver.

The service `Mem_30_LegacyAdapter_PropagateError()` does not cancel the underlying driver job, as in case of a vMem it cannot be canceled. Instead, the service sets a flag to return the job result `MEM_ECC_UNCORRECTED` when the job has finished on the driver.

The service `Mem_30_LegacyAdapter_BlankCheck()` is only available if it is supported by the underlying legacy driver. That is the case for all vMem drivers. In case of a Flash driver, the module checks whether the configuration parameter `FlsBlankCheckApi` as specified by AUTOSAR R24-11 exists and is set to true and only then forwards the call to the Flash driver's `BlankCheck` API. Otherwise, the service returns `E_MEM_SERVICE_NOT_AVAIL`. In case of an EEPROM driver, there is no `BlankCheck` API and the service always returns `E_MEM_SERVICE_NOT_AVAIL`.

3 Integration

3.1 Embedded Implementation

The module contains these source code files. Some files are generated by the configuration tool DaVinci Configurator Classic.

File Name	Description	Integration Tasks
Mem_30_LegacyAdapter.c	Source file of the module	-
Mem_30_LegacyAdapter.h	Header file that defines services used by typically upper layer modules.	-
Mem_30_LegacyAdapter_Types.h	Header file that defines general types and defines for development error reporting.	-
Mem_30_LegacyAdapter_LLAdapter.c	Source file that contains the adapter implementation parts to the underlying driver api.	-
Mem_30_LegacyAdapter_LLAdapter.h	Header file that declares interfaces of the adapter implementation parts to the underlying driver api.	-
Mem_30_LegacyAdapter_ErrorChecks_Int.h	Header file that contains the DET check functions of the Mem adapter itself.	-
Mem_30_LegacyAdapter_vMemAccM_Types.h	Header file that defines types, which were defined by vMemAccM in a legacy memory solution.	-
vMemAccM_vMemApi.h	Header file previously provided by vMemAccM, defines the interfaces of all functions which must be provided by the vMem modules so that the adapter can use them. All vMem instances must implement these functions.	-
Mem_30_LegacyAdapter_Cfg.h	Generated header file that contains the configuration parameters and access macros of the Mem adapter.	-
Mem_30_LegacyAdapter_Lcfg.c	Generated source file that contains configuration data of the Mem adapter.	-

Table 3.1 Source Code Files

3.2 Initialization

Before calling any functionality, the module has to be initialized by calling `Mem_30_LegacyAdapter_Init()`. `Mem_30_LegacyAdapter_Init()` will initialize all vMem instances, that are managed by the adapter. `Mem_30_LegacyAdapter_Init()` shall be called by the MemAcc.

**Note**

The Init of underlying Flash or EEPROM drivers will not be called by LegacyAdapter. Therefore, they must be initialized independently.

3.3 Main Functions

The module has one main function: `Mem_30_LegacyAdapter_MainFunction()`. `Mem_30_LegacyAdapter_MainFunction()` will call the main function of all vMem drivers managed by the adapter. `Mem_30_LegacyAdapter_MainFunction()` shall be called by the MemAcc.

**Note**

The MainFunction of underlying Flash or EEPROM drivers will not be called by LegacyAdapter. They must be called by another module to process the memory access requests.

3.4 Dynamic activation

The Mem_30_LegacyAdapter together with the referenced vMem(s) can be also dynamically activated and deactivated during runtime by the MemAcc. In this case the Mem_30_LegacyAdapter shall be configured within */MemGeneral/MemInvocation* as *INDIRECT_DYNAMIC*. This module together with the underlying vMem(s) should be then compiled and linked as a separated binary and must be loaded by the user later into a predefined RAM area to be executed.

It is important that the binary header of the Mem_30_LegacyAdapter is in the start of the image and the delimiter is in the end of the image, see also following figure:

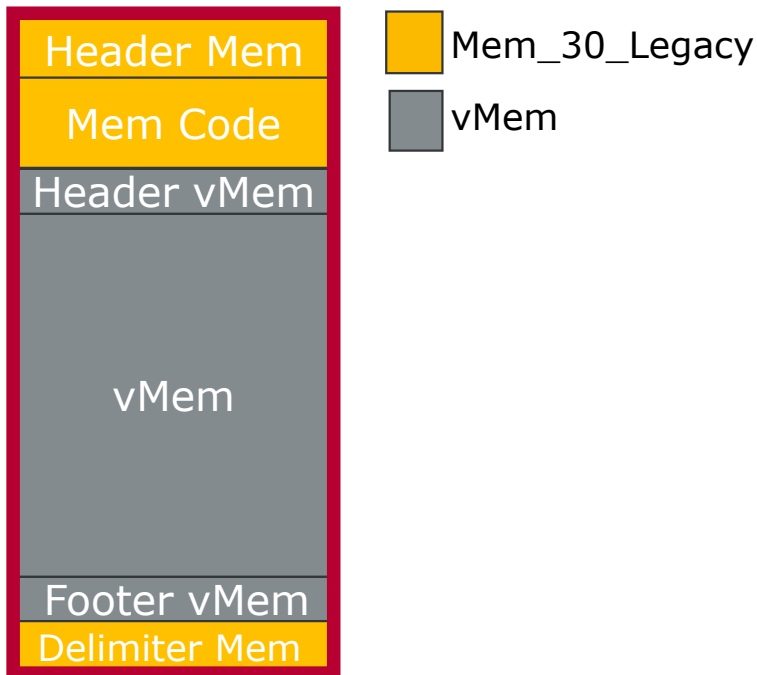


Figure 3.1 Binary image for legacy vMem activation

The type definition of the binary header is provided by MemAcc as `MemAcc_MemBinaryHeaderType` and consists of the actual header and the function pointer table (see `Mem_30_LegacyAdapter_FunctionPointerTable` structure within *Mem_30_LegacyAdapter.c*). The header itself consists of a unique id, header flags, and the address of the binary header as well as the address of the delimiter. The unique id and the flags are already set.



Caution

The actual address of the binary header and the delimiter must be provided by the integrator by defining `MEM_30_LEGACYADAPTER_HEADERADDRESS` as well as `MEM_30_LEGACYADAPTER_DELIMITERADDRESS` through the build environment.

Note that if a read only version should be provided as a binary image, the vMem(s) can be configured to be in a read only mode within `/vMem/vMemGeneral/vMemReadOnlyMode`. Relocatable Binaries as specified by AUTOSAR are not supported. For more general information see also [1], [3] and [4].

3.5 Synchronization

Synchronization between multiple flash memory users is done by the Memory Access module. Refer [3], for more details.

3.5.1 Critical Sections

No critical sections are used by the module.

3.6 Memory Sections

The Binary Header and the Delimiter are placed in own memory sections: *MEM_30_LEGACYADAPTER_START_SEC_CONST_HEADER_UNSPECIFIED* and *MEM_30_LEGACYADAPTER_START_SEC_CONST_DELIMITER_64*.

Special handling is only needed if the Mem_LegacyAdapter together with the vMem(s) should be dynamically activated and deactivated (see *Dynamic activation*).

3.7 Callouts

The module has no callout functions.

3.8 Error Reporting Det

The module has these properties used for the error reporting to the Det.

- > MEM_30_LEGACYADAPTER_MODULE_ID = 91
- > MEM_30_LEGACYADAPTER_VENDOR_ID = 30

4 API Description

4.1 Provided Functions

The module provides these functions.

Service functions are implemented by the module and are typically called by upper-layer modules. They are declared in `*.h`.

**Note**

In case an Flash or EEPROM driver is used, the Instance Id is used for a unique mapping from Mem Instance (instance of this module) to driver, but is not forwarded to Flash or EEPROM driver.

4.1.1 Mem_30_LegacyAdapter_BlankCheck

Prototype	
Std_ReturnType Mem_30_LegacyAdapter_BlankCheck (Mem_30_LegacyAdapter_InstanceIdType instanceId, Mem_30_LegacyAdapter_AddressType targetAddress, Mem_30_LegacyAdapter_LengthType length)	
Parameters	
instanceId [in]	ID of the related Mem instance.
targetAddress [in]	Blank check memory address.
length [in]	Length in bytes to blank check.
Return Codes	
E_MEM_SERVICE_NOT_AVAIL	The underlying driver does not support the blank check api.
E_OK	The requested job has been accepted and queued by the underlying driver.
E_NOT_OK	Otherwise.
Functional Description	
Asynchronous service to check if a page is blank in nv memory. This function triggers a job to check the erased state of the page which is referenced by targetAddress for the underlying driver associated with the requested memory instance. This is only done, if the underlying driver supports the blank check api.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.1 Mem_30_LegacyAdapter_BlankCheck

4.1.2 Mem_30_LegacyAdapter_DelInit

Prototype
void Mem_30_LegacyAdapter_DelInit ()
Functional Description
De-Initialization function De-Initialization is not supported by this module. It does nothing.
Particularities
Module is initialized.
Options
callcontext: TASK reentrant: FALSE synchronous: TRUE

Table 4.2 Mem_30_LegacyAdapter_DelInit

4.1.3 Mem_30_LegacyAdapter_Erase

Prototype	
Std_ReturnType Mem_30_LegacyAdapter_Erase (Mem_30_LegacyAdapter_InstanceIdType instanceId, Mem_30_LegacyAdapter_AddressType targetAddress, Mem_30_LegacyAdapter_LengthType length)	
Parameters	
instanceId [in]	ID of the related Mem instance.
targetAddress [in]	NV memory address to erase.
length [in]	Length in bytes to erase.
Return Codes	
E_OK	The requested job has been accepted and queued by the underlying driver.
E_NOT_OK	Otherwise.
Functional Description	
Asynchronous service to erase data from nv memory. This function triggers the service to erase data from the target nv memory for the underlying driver associated with the requested memory instance.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.3 Mem_30_LegacyAdapter_Erase

4.1.4 Mem_30_LegacyAdapter_GetJobResult

Prototype	
Mem_JobResultType Mem_30_LegacyAdapter_GetJobResult (Mem_30_LegacyAdapter_InstanceId- Type instancelId)	
Parameters	
instancelId [in]	ID of the related memory driver instance.
Return Codes	
MEM_JOB_OK	The last job has been finished successfully.
MEM_JOB_PENDING	A job is currently being processed.
MEM_JOB_FAILED	Job failed due to some unspecific reason.
MEM_INCONSISTENT	The checked page is not blank.
MEM_ECC_UNCORRECTED	Uncorrectable ECC errors occurred during memory access.
MEM_ECC_CORRECTED	Correctable ECC errors occurred during memory access.
Functional Description	
Checks and returns result of the most recent job for the requested memory instance.	
Options	
callcontext: TASK reentrant: TRUE synchronous: FALSE	

Table 4.4 Mem_30_LegacyAdapter_GetJobResult

4.1.5 Mem_30_LegacyAdapter_GetVersionInfo

Prototype	
void Mem_30_LegacyAdapter_GetVersionInfo (Std_VersionInfoType * versioninfo)	
Parameters	
versioninfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Functional Description	
Returns the version information Mem_30_LegacyAdapter_GetVersionInfo() returns version information, vendor ID and AUTOSAR module ID of the component.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.5 Mem_30_LegacyAdapter_GetVersionInfo

4.1.6 Mem_30_LegacyAdapter_HwSpecificService

Prototype	
Std_ReturnType Mem_30_LegacyAdapter_HwSpecificService (Mem_30_LegacyAdapter_InstanceIdType instancelId, Mem_30_LegacyAdapter_HwServiceIdType hwServiceId, Mem_30_LegacyAdapter_DataType * dataPtr, Mem_30_LegacyAdapter_LengthType * lengthPtr)	
Parameters	
instancelId [in]	ID of the related Mem instance.
hwServiceId [in]	Hardware-specific service request identifier for dispatching the request.
dataPtr [in,out]	Data pointer pointing to the job buffer. Value must not be NULL_PTR. If dataPtr is used by the hardware-specific service, the pointer must be valid until the job completed.
lengthPtr [in]	Size pointer of the data passed by dataPtr. Value must not be NULL_PTR.
Return Codes	
E_OK - The requested job has been accepted by the module. E_NOT_OK - The requested job has not been accepted by the module. E_MEM_SERVICE_NOT_AVAIL - The underlying Mem driver service function is not available.	
Functional Description	
Asynchronous service to trigger a hardware-specific job. This service is just a dispatcher to the hardware-specific service implementation referenced by the hwServiceId.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.6 Mem_30_LegacyAdapter_HwSpecificService

4.1.7 Mem_30_LegacyAdapter_Init

Prototype	
void Mem_30_LegacyAdapter_Init (Mem_30_LegacyAdapter_ConfigPtrType configPtr)	
Parameters	
configPtr [in]	Pointer to the configuration data.
Functional Description	
Initialization function Service to initialize this module. It initializes all variables, calls the init service of the underlying vMem drivers and sets the module state to initialized.	
Particularities	
Module is uninitialized.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.7 Mem_30_LegacyAdapter_Init

4.1.8 Mem_30_LegacyAdapter_MainFunction

Prototype
void Mem_30_LegacyAdapter_MainFunction ()
Functional Description
Triggers the main function of all managed vMem drivers.
Options
callcontext: TASK reentrant: FALSE synchronous: TRUE

Table 4.8 Mem_30_LegacyAdapter_MainFunction

4.1.9 Mem_30_LegacyAdapter_PropagateError

Prototype	
void Mem_30_LegacyAdapter_PropagateError (Mem_30_LegacyAdapter_InstanceIdType instancelId)	
Parameters	
instancelId [in]	ID of the related Mem instance.
Functional Description	
This service reports an access error in case this module cannot provide the access error information - typically for ECC faults. It is called by the system ECC handler to propagate an ECC error to the memory upper layers.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.9 Mem_30_LegacyAdapter_PropagateError

4.1.10 Mem_30_LegacyAdapter_Read

Prototype	
Std_ReturnType Mem_30_LegacyAdapter_Read (Mem_30_LegacyAdapter_InstanceIdType instanceId, Mem_30_LegacyAdapter_AddressType sourceAddress, Mem_30_LegacyAdapter_DataPtrType destinationDataPtr, Mem_30_LegacyAdapter_LengthType length)	
Parameters	
instanceId [in]	ID of the related memory driver instance.
sourceAddress [in]	Physical address to read data from.
destinationDataPtr [in]	Destination memory pointer to store the read data.
length [in]	Read length in bytes.
Return Codes	
E_OK	The requested job has been accepted and queued by the underlying driver.
E_NOT_OK	Otherwise.
Functional Description	
Read function. This function triggers a read job to copy data from the source address into the referenced destination data buffer for the underlying driver associated with the requested memory instance.	
Options	
callcontext: TASK reentrant: TRUE synchronous: FALSE	

Table 4.10 Mem_30_LegacyAdapter_Read

4.1.11 Mem_30_LegacyAdapter_Resume

Prototype	
Std_ReturnType Mem_30_LegacyAdapter_Resume (Mem_30_LegacyAdapter_InstanceIdType instanceld)	
Parameters	
instanceld [in]	ID of the related Mem instance.
Return Codes	
E_MEM_SERVICE_NOT_AVAIL	> The service is not supported by this module.
Functional Description	
This service is not supported by this module. It always returns E_MEM_SERVICE_NOT_AVAIL.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.11 Mem_30_LegacyAdapter_Resume

4.1.12 Mem_30_LegacyAdapter_Suspend

Prototype	
Std_ReturnType Mem_30_LegacyAdapter_Suspend (Mem_30_LegacyAdapter_InstanceIdType instanceld)	
Parameters	
instanceld [in]	ID of the related Mem instance.
Return Codes	
E_MEM_SERVICE_NOT_AVAIL	> The service is not supported by this module.
Functional Description	
This service is not supported by this module. It always returns E_MEM_SERVICE_NOT_AVAIL.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.12 Mem_30_LegacyAdapter_Suspend

4.1.13 Mem_30_LegacyAdapter_Write

Prototype	
Std_ReturnType Mem_30_LegacyAdapter_Write (Mem_30_LegacyAdapter_InstanceIdType instanceId, Mem_30_LegacyAdapter_AddressType targetAddress, Mem_30_LegacyAdapter_ConstDataPtrType sourceDataPtr, Mem_30_LegacyAdapter_LengthType length)	
Parameters	
instanceId [in]	ID of the related Mem instance.
targetAddress [in]	NV memory address to write to.
sourceDataPtr [in]	Application pointer to buffer with data to write to nv memory. Must stay valid until job is completed.
length [in]	Length in bytes to write.
Return Codes	
E_OK	The requested job has been accepted and queued by the underlying driver.
E_NOT_OK	Otherwise.
Functional Description	
Asynchronous service to write data to nv memory. This function triggers the service to write data from the source buffer to the target nv memory for the underlying driver associated with the requested memory instance.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.13 Mem_30_LegacyAdapter_Write

4.2 Required Functions

The module requires these functions from other modules. Refer to their documentation for API details.

Module	API
Det	Det_ReportError
vMem	vMem_Init vMem_Read vMem_Write vMem_Erase vMem_IsBlank vMem_GetJobResult vMem_MainFunction vMem_<HwSpecificService> ¹
Fls	Fls_Read Fls_Write Fls_Erase Fls_BlankCheck ² Fls_GetJobResult
Eep	Eep_Read Eep_Write Eep_Erase Eep_GetJobResult

Table 4.14 Required Functions

¹The vMem driver provides hardware-specific services, therefore the concrete api name can not be given here. The Mem_30_LegacyAdapter is able to call these services.

²Not all Flash drivers provide a BlankCheck API. The Mem_30_LegacyAdapter will only call it if available, see also *Compliance*.

4.3 Service Ports

This module does not provide service ports.

5 Configuration

The functionalities of the module are configured with DaVinci Configurator Classic.

For details on configuration options refer to the module-specific description that is shown in DaVinci Configurator Classic.

Each Mem instance (instance of this module) references a *vMemInstance* or *FlsConfigSet* or *EepInitConfiguration* of a legacy memory driver. Applicable parameters are synced to the configuration with validators. Therefore, it is recommended to first configure the underlying legacy memory driver and then set the reference from Mem instance to *vMemInstance* or *FlsConfigSet* or *EepInitConfiguration*. However, some parameters have to be configured manually, as they are not present in the legacy memory drivers. See section *Additional Configuration Tasks*.

Note, that for a referenced *vMemInstance* or *FlsConfigSet*, there will always be created automatically as many *MemSectorBatch* container as there are *vMemSector* container or *FlsSector* container. In case of a referenced *EepInitConfiguration* there will be precisely one *MemSectorBatch* container and *MemNumberOfSectors* is determined automatically from *EepEraseUnitSize* and *EepSize*.

5.1 Configuration Variants

This module supports these configuration variants:

> VARIANT-PRE-COMPILE

The configuration classes of the parameters depend on the supported configuration variants. For their definitions please see the module-specific BSWMD ARXML file.

5.2 Additional Configuration Tasks

The legacy vMem/Fls/Eep drivers do not specify the following configuration parameters:

- > /MICROSAR/Mem_30_LegacyAdapter/Mem/MemInstance/MemSectorBatch/MemBurstSettings/MemMaxReadSize
- > /MICROSAR/Mem_30_LegacyAdapter/Mem/MemInstance/MemSectorBatch/MemSpecifiedEraseCycles
- > /MICROSAR/Mem_30_LegacyAdapter/Mem/MemPublishedInformation/MemErasedValue

Therefore, they have to be configured manually within LegacyAdapter.

MemMaxReadSize:

This value is the maximum number of bytes the MemAcc module will request within one Mem read request. The parameter shall be used to improve performance. If no *MemBurstSettings* are configured, MemAcc will use *MemMinReadSize* to split the read request into small job steps.

**Caution**

The *MemMaxReadSize* parameter is placed by AUTOSAR within the optional *MemBurstSettings* container. For performance reasons it is strongly recommended to set this value. Therefore, a *MemBurstSettings* container will be created automatically for each *MemSectorBatch* container by validators. If *MemMaxReadSize* is not set to a reasonably high value, many MainFunction cycles might be needed for read requests.

MemSpecifiedEraseCycles:

Enter the number of erase cycles specified for the memory device you are using. Typically, this information is given in the device hardware manual.

MemErasedValue:

This parameter specifies the content of an erased memory cell. Typically it is specified in the hardware manual of the device that is being used. FIs and Eep drivers might specify an erased value in their respective published information. However, the LegacyAdapter does not validate these values and does not sync them to the *MemErasedValue*.

**Caution**

It is crucial that the erased value is the same for all referenced legacy drivers as upper layers rely on this value.

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
Bank	Group of sector batches
DaVinci Configurator	Generation tool for MICROSAR Classic modules
Device	Group of banks
Page (Write Page)	Smallest writeable unit of a memory device (in bytes)
Page Burst	Aggregated access of memory pages for improved performance
Sector	Smallest erasable memory unit (in bytes)
Sector Batch	Aggregation of sectors with uniform size
Sector Burst	Aggregation of sectors for improved erase performance

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DET	Development Error Tracer
DFLS	Data-Flash
ECU	Electronic Control Unit
EEP	EEPROM
EXFLS	External Flash
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
MEMACC	Memory Access
PFLS	Program-Flash
SWS	Software Specification

Table 6.2 [Abbreviations]

7 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com